Programming Assignment 2 | Team 14

Andrew Goodson, Elijah Johnson, Sophie Shah, Nicholas Lindner

Due Friday, March 27, 12:50 PM on Canvas

Software Design

This system uses a TCP-based client-server architecture where the client initiates all communication and the server responds accordingly.
The client is responsible for establishing a socket connection, sending user-input commands with an ObjectOutputStream, receiving responses with an ObjectInputStream, downloading files when requested, and measuring the RTT for file transfers.
The server is responsible for listening for incoming client connections through ServerSocket, handling one client connection at a time, processing the client requests, and sending files or error messages.
After connection, the server sends a "Hello" message and the client then sends a file name. If the file exists, the server sends the file size and then the file data. If the file doesn't exist, the server responds with "File not found". The client saves the received files in a local directory and displays the download progress. For termination, the client sends "bye" and the server responds with "disconnected", and both programs exit gracefully.

Round-Trip Time Statistics at the Client

Statistic	Time (ms)
Mean	1003
Median	1018
Mode	N/A
Minimum	491
Maximum	1350
Standard Deviation	318.07

Comparison with PA1 RTT Statistics

Statistic	Time (ms)
Mean	26.4
Median	17
Mode	17
Minimum	1
Maximum	74
Standard Deviation	23.75

Key Differences:

1. Higher RTT in PA2, likely due to this assignment including file transfers instead of simple messages. A larger data payload leads to increased transmission time.
2. Increased variability due to I/O delays in the disk and buffering effects. This lead to a higher standard deviation compared to PA1.
3. Distribution skewed to higher values due to file sizes. PA1 had a lower median and mode because of its lightweight communication.
4. Measured latency and throughput effects with PA2 as compared to measuring mostly latency with PA1.

Lessons Learned

We learned a lot about TCP, file transfers, and error handling. This programming assingment truly showed how TCP has reliable communication at the cost of slower RTT. Adding overhead with acknowledgements and retransmissions in TCP can cause slower communications between the client and server. We learned how file transfers require strong protocol design, such as how sending the file size first is critical to prevent incomplete reads. We also learned the importance of error handling. Handling cases for an unknown host, missing files, and connection refusals can improve the robustness of the network system.